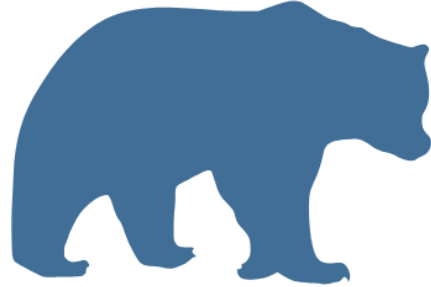


Introducción a la librería Pandas



¿Qué es pandas?

Pandas es una librería de Python de código abierto especializada en el manejo y análisis de estructuras de datos. Es rápida, potente, flexible y fácil de usar.

Tiene el objetivo de convertirse en la herramienta de análisis/manipulación de datos de código abierto más potente y flexible.

Características

- Define **nuevas estructuras de datos** basadas en los arrays de NumPy pero con nuevas funciones: Series, DataFrames y Panel.
- Permite **leer y escribir** fácilmente **ficheros** en formato CSV, Excel y bases de datos SQL.
- Permite **acceder** a los datos **mediante índices o nombres** para filas y columnas.
- Ofrece métodos para **reordenar, dividir y combinar** conjuntos de datos.
- Permite trabajar con **series temporales** (indexadas por un timestamp/marca temporal).
- Realiza todas estas operaciones de manera muy **eficiente**.

Estructuras de datos de pandas

Convenio de importación de Pandas (recordamos también el de Numpy)

```
import pandas as pd
import numpy as np
```

Pandas dispone de dos estructuras de datos principales:

- **Series:** Estructura de una dimensión.
- **DataFrame:** Estructura de dos dimensiones (tablas).

Estas estructuras se construyen a partir de arrays de la librería NumPy, añadiendo nuevas funcionalidades.

Series

Una serie es un objeto unidimensional de estilo array que contiene una secuencia de valores del mismo tipo y un array asociado de etiquetas de datos, que corresponde a su índice. Las series son homogéneas, es decir, sus elementos tienen que ser del mismo tipo, y su tamaño es inmutable, no se puede cambiar, aunque sí su contenido.

Así sería la creación de una serie con índice, que identifique cada punto de datos con una etiqueta:

```
obj = pd.Series([4, 7, -5, 3], index=["d","b","a","c"])
```

Ahora, podemos acceder a los elementos de la serie usando los índices:

```
print(obj["a"]) #Mostrará el valor -5

obj["d"] = 6
print(obj) #Mostrará lo que pongo a continuación
```

d	6
b	7
a	-5
c	3
dtype: int64	

En este caso, ["d","b","a","c"] se interpreta como lista de **índices** aunque contenga cadena de texto.

El objeto Series más sencillo que se puede crear es uno formado sólo por un array de datos.

```
obj = pd.Series([4, 7, -5, 3])
```

Si el índice no se especifica como en este caso, se crea uno predeterminado formado por los enteros 0 a N-1 (siendo N el número de elementos de la serie).

Métodos para ver las características de una serie:

- **s.size**: Devuelve el número total de elementos de la serie s.
- **s.index**: Devuelve un objeto index que representa las etiquetas de los elementos de s.
- **s.dtype** : Devuelve el tipo de datos de los elementos de la serie s.

Otra forma de ver las series podría ser la de un **diccionario ordenado de longitud fija**, ya que se asignan índices a datos.

Crear serie a partir de diccionario
<pre>dic = {"Ohio": 35000, "Texas": 71000, "Oregon": 16000, "Utah": 5000} new_series = pd.Series(dic)</pre>
Crear diccionario a partir de serie
<pre>new_dic = new_series.to_dict()</pre>

Si queremos crear una serie a partir de un diccionario pero modificando las claves, se puede hacer de la siguiente forma

Crear serie a partir de diccionario cambiando índices
<pre>#Tenemos este diccionario dic = {"Ohio": 35000, "Texas": 71000, "Oregon": 16000, "Utah": 5000} #Queremos que la serie tenga este nuevo índice estados = ["California", "Ohio", "Oregon", "Texas"] new_series_modif = pd.Series(dic, index=estados)</pre>

```
print(new_serie_modif)
```

```
California    NaN
Ohio          35000.0
Oregon        16000.0
Texas         71000.0
dtype: float64
```

En esta serie resultante tendremos “California” con un valor **NaN** asociado (Not a Number), que es la expresión que **pandas** usa para indicar de un **valor ausente**. Por otro lado, “Utah”, que sí estaba en el diccionario, no aparecerá en la nueva serie porque en el índice no estaba ese valor, así que **se excluye**.

Para saber si en una serie hay valores ausentes/nulos/faltantes se pueden emplear las funciones **isna** y **notna**

```
print(pd.isna(new_serie_modif))
```

```
California    True
Ohio          False
Oregon        False
Texas         False
dtype: bool
```

```
print(pd.notna(new_serie_modif))
```

```
California    False
Ohio          True
Oregon        True
Texas         True
dtype: bool
```

Por último, se le puede poner un nombre, tanto al objeto Series como al índice, con el atributo **name**

```
dic = {"Pedro": 183, "Ana": 171, "Raquel": 157, "Nico": 165}
```

```
new_serie = pd.Series(dic)
```

```

new_serie.name = "Altura_cm"
new_serie.index.name = "Nombres"

print(new_serie)

```

Nombres	
Pedro	183
Ana	171
Raquel	157
Nico	165

```

Name: Altura_cm, dtype: int64

```

DataFrame

Un **dataframe** representa una tabla rectangular de datos y contiene una colección de columnas ordenada y con nombre. Cada columna puede tener un tipo de valor distinto (numérico, cadena de texto, booleano...). El objeto dataframe tiene un índice para las filas y otro para las columnas. Puede verse como un conjunto de objetos Series que comparten el mismo índice.

```

import pandas as pd

datos = {"ciudad":["Almería", "Zafra", "Madrid", "Salamanca"],
        "temp_max":[38, 40, 36, 32],
        "temp_min":[12, -2, -8, -5]}

frame = pd.DataFrame(datos)

frame.index.name = "Indice"
frame.columns.name = "Características"

print(frame)

```

	Características	ciudad	temp_max	temp_min
Indice				
0		Almería	38	12
1		Zafra	40	-2
2		Madrid	36	-8
3		Salamanca	32	-5

Como vemos en el ejemplo anterior, el índice de las filas se asigna de manera automática (tal y como pasa con las series si no se especifica). Las columnas se ordenan según están en la declaración de datos, a no ser que se especifique una secuencia concreta como en el ejemplo siguiente:

```
import pandas as pd
```

```
datos = {"ciudad": ["Almería", "Zafra", "Madrid", "Salamanca"],  
        "temp_max": [38, 40, 36, 32],  
        "temp_min": [12, -2, -8, -5]}
```

```
frame = pd.DataFrame(datos, columns=["temp_min", "temp_max",  
"ciudad"])
```

```
print(frame)
```

	temp_min	temp_max	ciudad
0	12	38	Almería
1	-2	40	Zafra
2	-8	36	Madrid
3	-5	32	Salamanca

#Si se indica una columna que no está en el diccionario, se rellena con valores faltantes

```
frame2 = pd.DataFrame(datos, columns=["temp_min", "humedad",  
"ciudad"])
```

```
print(frame2)
```

	temp_min	humedad	ciudad
0	12	NaN	Almería
1	-2	NaN	Zafra
2	-8	NaN	Madrid
3	-5	NaN	Salamanca

A diferencia de las series, los dataframes no tienen atributo name (aunque si podemos ponerle nombre al índice de las filas y columnas como hemos visto en el ejemplo previo).

Algunos métodos útiles para dataframes:

```
frame.head() #Devuelve las primeras cinco filas  
frame.tail() #Devuelve las últimas cinco filas
```

```

frame["nombre_columna"] #Devuelve la columna "nombre_columna" como una serie
frame.nombre_columna    #Devuelve la columna "nombre_columna" como una serie
                        (esta segunda forma de consultar columna no funciona
                        siempre, depende del nombre de la columna)
frame.loc["nombre_fila"] #Devuelve la fila con indice = "nombre_fila"
frame.iloc[x]           #Devuelve la fila de la posición x

del frame["nombre_columna"] #Elimina columna "nombre_columna"

frame.T                 #Traspone el dataframe (intercambia filas por columnas)

frame.to_numpy          #Convierte el DataFrame en un numpy array.

```

Podemos modificar el contenido de las columnas dándole el mismo valor a todas las filas por medio de la asignación, o variando el contenido empleando arrays.

```

import pandas as pd
import numpy as np

datos = {"nombre":["Jose", "Juan", "Carlos", "Salvador"],
         "estado_civil":["casado", "viudo", "soltero", "casado"],
         "edad":[32, 83, 49, 64]}

frame = pd.DataFrame(datos, columns=["nombre", "estado_civil",
                                     "hijos"])

print(frame)

           nombre estado_civil hijos
0         Jose      casado    NaN
1         Juan      viudo    NaN
2        Carlos    soltero    NaN
3    Salvador      casado    NaN

#Asignamos mismo valor por asignación
frame["hijos"] = 1

print(frame)

```

	nombre	estado_civil	hijos
0	Jose	casado	1
1	Juan	viudo	1
2	Carlos	soltero	1
3	Salvador	casado	1

```
#Asignamos array de valores con longitud = num.filas del dataframe
frame["hijos"] = np.arange(4.)
```

```
print(frame)
```

	nombre	estado_civil	hijos
0	Jose	casado	0.0
1	Juan	viudo	1.0
2	Carlos	soltero	2.0
3	Salvador	casado	3.0

```
#Asignamos serie, solo se asigna para los indices existentes
val = pd.Series([1,2,4,3], index=["primero", 3, "segundo", "tercero"])
```

```
frame["hijos"] = val
print(frame)
```

	nombre	estado_civil	hijos
0	Jose	casado	NaN
1	Juan	viudo	NaN
2	Carlos	soltero	NaN
3	Salvador	casado	2.0

```
#Asignar columna que no existe crea una nueva
#OJO!! no se pueden crear con frame.nueva_columna
frame["mascotas"] = frame["hijos"]
print(frame)
```

	nombre	estado_civil	hijos	mascotas
0	Jose	casado	NaN	NaN
1	Juan	viudo	NaN	NaN
2	Carlos	soltero	NaN	NaN
3	Salvador	casado	2.0	2.0

Para terminar con los dataframes, vamos a comentar la posibilidad de crearlos a partir de diccionarios anidado de diccionarios


```
import pandas as pd
import numpy as np

poblaciones = {"Ohio": {2000:1.5, 2001:1.7, 2002:3.6},
               "Nevada": {2001:1.4, 2002:2.9}}

frame = pd.DataFrame(poblaciones)

print(frame)
```

```
      Ohio  Nevada
2000    1.5     NaN
2001    1.7     1.4
2002    3.6     2.9
```

TABLA RESUMEN: Posibles entradas de datos para constructor DataFrame

Tipo de estructura de datos	Conversión a DataFrame	Notas
ndarray de dos dimensiones	Se convierte directamente en un DataFrame	Puede incluir etiquetas opcionales para filas y columnas
Diccionario de arrays, listas o tuplas	Cada secuencia se convierte en una columna	Todas deben tener la misma longitud
Diccionario de Series	Cada Series se convierte en una columna	Los índices de las Series se combinan para formar el índice de las filas final (a no ser que se pase un índice explícito)
Diccionario de diccionarios	Cada diccionario interno se convierte en una columna	Las claves internas se convierten en índices de filas

Objetos índice

Los objetos índice de pandas albergan etiquetas de ejes (incluyendo el nombre de las columnas de un dataframe) y otros metadatos (como los nombres de los ejes). Cualquier array u otra secuencia de etiquetas que se utilice al construir una serie o dataframe se convierte internamente en un índice. Estos objetos son **inmutables**, por lo que no pueden ser modificados por el usuario.

```
import pandas as pd
import numpy as np
```

```
obj = pd.Series(np.arange(3), index=["a", "b", "c"])
index = obj.index

index[1]="d"

index[1]="d"
~~~~~^^^
File ~~~~~
raise TypeError("Index does not support mutable o
TypeError: Index does not support mutable operations
```

Un índice de pandas se comporta como un conjunto de tamaño fijo, pero que además **puede contener etiquetas duplicadas**. Si hacemos selecciones de la etiqueta que aparece varias veces, se incluirán en la selección todas las apariciones de dicha etiqueta.

Métodos y propiedades del objeto índice:

Método / propiedad	Descripción
append()	Concatena con objetos Index adicionales, produciendo un nuevo índice.
difference()	Calcula la diferencia de conjuntos como un Index. Retorna los elementos que están en el índice original pero no en el índice pasado como argumento.
intersection()	Calcula la intersección entre conjuntos de índices y devuelve los elementos comunes.
union()	Retorna la unión de dos Index, combinando los elementos sin duplicados.
isin()	Devuelve un array booleano indicando si cada elemento del Index está presente en una lista u otro Index.
delete()	Devuelve un nuevo Index con un elemento eliminado en la posición especificada.
drop()	Devuelve un nuevo Index con ciertos valores eliminados. Funciona con valores en lugar de posiciones.
insert()	Inserta un nuevo valor en una posición específica dentro del Index, devolviendo un nuevo Index.
is_monotonic	Propiedad booleana que indica si los valores del Index están ordenados de forma creciente o decreciente.

is_unique	Propiedad booleana que indica si todos los valores en el Index son únicos (sin duplicados).
unique()	Retorna un array con los valores únicos en el Index.

Índice de contenidos

FUNCIONALIDAD ESENCIAL.....	2
Reindexación.....	2
Eliminar entradas de un eje.....	3
Indexación, selección y filtrado.....	5
Aritmética y alineación de datos.....	8
Aplicación y asignación de funciones.....	11
Ordenación y asignación de rangos.....	12
Índices de ejes con etiquetas duplicadas.....	16
 RESUMIR Y CALCULAR ESTADÍSTICAS DESCRIPTIVAS.....	 17
Métodos de estadística descriptivas y de resumen:.....	18
Métodos únicos, de recuentos de valores y de pertenencia a conjuntos.....	19

FUNCIONALIDAD ESENCIAL

En este apartado veremos la mecánica fundamental de la interacción con los datos contenidos en una serie o dataframe.

Reindexación

Consiste en crear un nuevo objeto con los valores reordenados para que se alineen con el nuevo índice. Se hace llamando a la función **reindex**.

```
import pandas as pd
import numpy as np

# Reindex con Series
obj = pd.Series([4.5, 7.2, -5.3, 3.6], index=["d", "b", "a", "c"])
print(obj)
```

d	4.5
b	7.2
a	-5.3
c	3.6

dtype: float64

```
obj2 = obj.reindex(["a", "b", "c", "d", "e"])
#Vemos que el nuevo índice "e" lo rellena con NaN
print(obj2)
```

a	-5.3
b	7.2
c	3.6
d	4.5
e	NaN

dtype: float64

```
# Reindex con Dataframes
frame = pd.DataFrame(np.arange(9).reshape(3,3),
                     index=["a", "c", "d"],
                     columns=["Ohio", "Texas", "California"])
print(frame)
```

	Ohio	Texas	California
a	0	1	2
c	3	4	5
d	6	7	8

```

states = ["Texas", "Utah", "California"]
frame2 = frame.reindex(states, axis=1) #igual que axis="columns"

print(frame2)

```

	Texas	Utah	California
a	1	NaN	2
c	4	NaN	5
d	7	NaN	8

Argumentos que podemos usar con la función reindex:

Argumento	Descripción
labels	Nueva secuencia a utilizar como índice. Puede ser cualquier estructura de datos de tipo secuencia.
index	Usa la secuencia pasada como nuevas etiquetas de índice
columns	Usa la secuencia pasada como nuevas etiquetas de columna
axis	Determina el eje a reindexar: index (axis=0, por defecto) o columns (axis=1)
method	Para rellenar los valores faltantes cuando se agregan nuevas etiquetas en filas o columnas: "ffill" rellena hacia delante y "bfill" rellena hacia atrás
fill_value	Determina el valor a utilizar para rellenar valores faltantes. El valor predeterminado es "missing" que rellena con valores nulos
limit	Al rellenar valores, determina el número máximo de elementos a rellenar
copy	Si es True, copia los datos subyacentes incluso aunque el nuevo índice sea equivalente al antiguo. Si es False, no copia los datos cuando los índices sean equivalentes

Eliminar entradas de un eje

Se puede hacer de varias formas, empleando la reindexación definiendo un índice que no tenga la entrada que no queremos, o con el método drop.

```

import pandas as pd
import numpy as np

obj = pd.Series(np.arange(5.), index=["a", "b", "c", "d", "e"])
print(obj)

a    0.0
b    1.0
c    2.0
d    3.0
e    4.0
dtype: float64

# Método drop
obj2 = obj.drop(["c", "e"])
print(obj2)

a    0.0
b    1.0
d    3.0
dtype: float64

# Con reindex
nuevo_indice = ["a", "b", "d"]
obj3 = obj.reindex(nuevo_indice)
print(obj3)

a    0.0
b    1.0
d    3.0
dtype: float64

```

Con los dataframes, podemos quitar elementos de cualquier eje.

```

import pandas as pd
import numpy as np

frame = pd.DataFrame(np.arange(16).reshape(4,4),
                      index=["Ohio", "Texas", "California", "Utah"],
                      columns=["Uno", "Dos", "Tres", "Cuatro"])

print(frame)

```

	Uno	Dos	Tres	Cuatro
Ohio	0	1	2	3
Texas	4	5	6	7
California	8	9	10	11
Utah	12	13	14	15

```
frame2 = frame.drop(index="Ohio", columns=["Cuatro", "Dos"])
print(frame2)
```

	Uno	Tres
Texas	4	6
California	8	10
Utah	12	14

Indexación, selección y filtrado

Tanto en las Series como en los Dataframes se pueden emplear corchetes o los operadores **loc** e **iloc** (preferiblemente usar estos dos últimos operadores).

```
import pandas as pd
import numpy as np

# Trabajamos con series
obj = pd.Series(np.arange(4.), index=["a", "b", "c", "d"])
print(obj)

      a    0.0
      b    1.0
      c    2.0
      d    3.0

print(obj[obj < 2])

      a    0.0
      b    1.0

print(obj.loc[["a", "b"]])

      a    0.0
      b    1.0
```



```
#Serie con índice de enteros
```

```
obj2 = pd.Series(np.arange(3.), index=[2, 0, 1])
```

```
print(obj2)
```

```
2    0.0
0    1.0
1    2.0
dtype: float64
```

```
#Observa la diferencia entre iloc y [] con el índice de enteros
```

```
print(obj2.iloc[[0, 1, 2]])
```

```
2    0.0
0    1.0
1    2.0
```

```
print(obj2[[0, 1, 2]]) #trata los enteros como etiquetas
```

```
0    1.0
1    2.0
2    0.0
```

```
# Trabajamos con Dataframes
```

```
frame = pd.DataFrame(np.arange(16).reshape(4,4),
```

```
                    index=["Ohio", "Texas", "California", "Utah"],
```

```
                    columns=["Uno", "Dos", "Tres", "Cuatro"])
```

```
print(frame)
```

	Uno	Dos	Tres	Cuatro
Ohio	0	1	2	3
Texas	4	5	6	7
California	8	9	10	11
Utah	12	13	14	15

```
print(frame[["Dos", "Tres"]])
```

	Dos	Tres
Ohio	1	2
Texas	5	6
California	9	10
Utah	13	14

```
print(frame > 5)
```

	Uno	Dos	Tres	Cuatro
Ohio	False	False	False	False
Texas	False	False	True	True
California	True	True	True	True
Utah	True	True	True	True

```
print(frame[frame["Uno"] > 5])
```

```

      Uno  Dos  Tres  Cuatro
California  8   9   10    11
Utah       12  13   14    15

```

```
print(frame[:2])
```

```

      Uno  Dos  Tres  Cuatro
Ohio    0   1   2     3
Texas   4   5   6     7

```

```
print(frame.loc["Texas", ["Uno", "Dos"]])
```

```

      Uno  4
      Dos  5
      Name: Texas, dtype: int64

```

```
print(frame.iloc[[1, 2], [3, 0, 1]])
```

```

      Cuatro  Uno  Dos
Texas         7   4   5
California    11   8   9

```

```
print(frame.iloc[:, :3][frame.Uno < 7])
```

```

      Uno  Dos  Tres
Ohio    0   1   2
Texas   4   5   6

```

[:, :3] todas las filas y las primeras 3 columnas

Opciones de indexado con objetos Dataframe

Tipo	Descripción
df[column]	Selecciona una columna o secuencia de columnas del dataframe
df.loc[rows]	Selecciona una fila o subconjunto de filas por etiqueta
df.loc[:,cols]	Selecciona una sola columna o conjunto de columnas por etiqueta
df.loc[rows,cols]	Selecciona una fila (o filas) y columna (o columnas) por etiqueta
df.iloc[rows]	Selecciona una fila o subconjunto de filas por posición de entero
df.iloc[:,cols]	Selecciona una sola columna o conjunto de columnas por posición de entero
df.iloc[rows,cols]	Selecciona una fila (o filas) y columna (o columnas) por posición de entero
df.at[row,col]	Selecciona un solo valor escalar por etiqueta de fila y columna
df.iat[row,col]	Selecciona un solo valor escalar por posición de fila y columna (enteros)
método reindex	Selecciona filas o columnas por etiquetas

Aritmética y alineación de datos

Pandas simplifica mucho el trabajo con objetos que tienen distinto índice. Por ejemplo, al sumar dos objetos con índices distintos, el resultado incluirá la unión de todos los índices. La alineación interna de datos introduce valores faltantes (NaN) en las ubicaciones de estos casos en los que los índices difieren. Los valores faltantes se propagarán en los cálculos aritméticos posteriores.

Métodos aritmeticos:

Método	Descripción
add, radd	Métodos para suma (+)
sub, rsub	Métodos para resta (-)
div, rdiv	Métodos para división (/). Devuelve el resultado de la división incluyendo la parte fraccionaria
floordiv, rfloordiv	Métodos para división por el suelo o de piso (/). Devuelve el cociente de la división.
mul, rmul	Métodos para multiplicación (*)
pow, rpow	Métodos para exponencial (**)

Cada método tiene un equivalente que comienza con **r** que hace lo mismo pero con los argumentos invertidos, es decir, `obj1.rdiv(obj2)` es lo mismo que `obj2.div(obj1)`.

Puede ser que nos interese rellenar con un valor diferente al NaN cuando se encuentra una etiqueta de un eje en un objeto pero no en el otro. Para ello se emplea el argumento **fill_value**.

```
import pandas as pd
import numpy as np

frame1 = pd.DataFrame(np.arange(12.).reshape(3,4),
                      columns=list("abcd"))
```

	a	b	c	d
0	0.0	1.0	2.0	3.0
1	4.0	5.0	6.0	7.0
2	8.0	9.0	10.0	11.0

```
frame2 = pd.DataFrame(np.arange(20.).reshape(4,5),
                      columns=list("abcde"))
```

	a	b	c	d	e
0	0.0	1.0	2.0	3.0	4.0
1	5.0	6.0	7.0	8.0	9.0
2	10.0	11.0	12.0	13.0	14.0
3	15.0	16.0	17.0	18.0	19.0

```
sumal = frame1 + frame2
print(sumal)
```

	a	b	c	d	e
0	0.0	2.0	4.0	6.0	NaN
1	9.0	11.0	13.0	15.0	NaN
2	18.0	20.0	22.0	24.0	NaN
3	NaN	NaN	NaN	NaN	NaN

```
suma = frame1.add(frame2, fill_value=0)
print(suma)
```

	a	b	c	d	e
0	0.0	2.0	4.0	6.0	4.0
1	9.0	11.0	13.0	15.0	9.0
2	18.0	20.0	22.0	24.0	14.0
3	15.0	16.0	17.0	18.0	19.0

```
division = frame1.div(frame2)
print(division)
```

	a	b	c	d	e
0	NaN	1.000000	1.000000	1.000000	NaN
1	0.8	0.833333	0.857143	0.875000	NaN
2	0.8	0.818182	0.833333	0.846154	NaN
3	NaN	NaN	NaN	NaN	NaN

```
division2 = frame1.rdiv(frame2)
print(division2)
```

	a	b	c	d	e
0	NaN	1.000000	1.000000	1.000000	NaN
1	1.25	1.200000	1.166667	1.142857	NaN
2	1.25	1.222222	1.200000	1.181818	NaN
3	NaN	NaN	NaN	NaN	NaN

También se pueden hacer operaciones entre Series y Dataframes, haciendo coincidir el índice de la serie con las columnas del dataframe (en caso de que coincidan las filas, se debe especificar en el argumento del método aritmético que se use).

```
import pandas as pd
import numpy as np

frame = pd.DataFrame(np.arange(12.).reshape(4,3),
                      columns=list("bde"),
                      index=["Utah", "Texas", "Ohio", "Oregon"])

print(frame)
```

	b	d	e
Utah	0.0	1.0	2.0
Texas	3.0	4.0	5.0
Ohio	6.0	7.0	8.0
Oregon	9.0	10.0	11.0

```
serie = frame.iloc[0]
print(serie)
```

b	0.0
d	1.0
e	2.0

Name: Utah, dtype: float64

```
resta = frame - serie
print(resta)
```

	b	d	e
Utah	0.0	0.0	0.0
Texas	3.0	3.0	3.0
Ohio	6.0	6.0	6.0
Oregon	9.0	9.0	9.0

```
# Ejemplo para ver cómo sería si lo que coinciden son las filas
serie1 = frame["d"]
```

Utah	1.0
Texas	4.0
Ohio	7.0
Oregon	10.0

Name: d, dtype: float64

```
resta2 = frame.sub(serie1, axis=0) #o axis="index"
print(resta2)
```

	b	d	e
Utah	-1.0	0.0	1.0
Texas	-1.0	0.0	1.0
Ohio	-1.0	0.0	1.0
Oregon	-1.0	0.0	1.0

Aplicación y asignación de funciones

Cuando se aplican unfuncs (universal functions) de NumPy a objetos de Pandas, se comportan de manera vectorizada, es decir, operan elemento por elemento sin necesidad de usar bucles explícitos:

```
import pandas as pd
import numpy as np

frame = pd.DataFrame(np.random.standard_normal((4,3)),
                     columns=list("bde"),
                     index=["Utah", "Texas", "Ohio", "Oregon"])

print(frame)
```

	b	d	e
Utah	0.307882	0.413931	-0.001261
Texas	0.285069	0.198186	-0.656558
Ohio	0.376168	-2.194243	-0.639403
Oregon	0.242880	0.522610	0.545270

```
serie = np.abs(frame.iloc[0])
```

b	0.152555
d	0.648521
e	0.176412

Name: Utah, dtype: float64

```
print(np.sqrt(serie))
```

b	0.390583
d	0.805308
e	0.420014

Name: Utah, dtype: float64

También se pueden aplicar funciones a cada columna o fila con el método `apply`.

```
import pandas as pd
import numpy as np

# Calcula la diferencia entre el max y el min de una serie
def f1(x):
    return x.max() - x.min()

frame = pd.DataFrame(np.random.standard_normal((4,3)),
                      columns=list("bde"),
                      index=["Utah", "Texas", "Ohio", "Oregon"])
print(frame)
```

	b	d	e
Utah	0.025398	0.294115	1.508486
Texas	-0.422561	-0.844549	0.765746
Ohio	0.548420	-0.478634	0.226512
Oregon	-0.636527	0.017326	0.944349

```
# La función f1 se invoca una vez para cada columna
print(frame.apply(f1))
```

	b	d	e
	1.184947	1.138664	1.281975

dtype: float64

```
# La función f1 se invoca una vez para cada fila
print(frame.apply(f1, axis="columns"))
```

	Utah	Texas	Ohio	Oregon
	1.483088	1.610295	1.027053	1.580877

dtype: float64

Ordenación y asignación de rangos

Ordenar un conjunto de datos según un cierto criterio es una operación interna importante. Para ordenar lexicográficamente por etiqueta de fila o columna, se emplea el método **sort_index**, que devuelve un objeto nuevo y ordenado. Para ordenar por sus valores se emplea **sort_values**.

```
import pandas as pd
```

```
import numpy as np
frame = pd.DataFrame(np.random.standard_normal((4,4)),
                      columns=list("dabc"),
                      index=["Trenes", "Barcos", "Autobus", "Coche"])
```

	d	a	b	c
Trenes	-0.697110	1.750307	-0.594645	0.685423
Barcos	0.410974	-0.973952	-1.070031	1.308861
Autobus	1.039543	-1.622006	-1.249022	-0.926627
Coche	-0.505677	0.353713	0.625704	2.598641

```
# Ordenamos por filas
print(frame.sort_index())
```

	d	a	b	c
Autobus	1.039543	-1.622006	-1.249022	-0.926627
Barcos	0.410974	-0.973952	-1.070031	1.308861
Coche	-0.505677	0.353713	0.625704	2.598641
Trenes	-0.697110	1.750307	-0.594645	0.685423

```
# Ordenamos por columnas
print(frame.sort_index(axis=1)) # o axis="columns"
```

	a	b	c	d
Trenes	1.750307	-0.594645	0.685423	-0.697110
Barcos	-0.973952	-1.070031	1.308861	0.410974
Autobus	-1.622006	-1.249022	-0.926627	1.039543
Coche	0.353713	0.625704	2.598641	-0.505677

```
# Ordenamos por columnas en orden inverso
print(frame.sort_index(axis=1, ascending=False)) # o axis="columns"
```

	d	c	b	a
Trenes	-0.697110	0.685423	-0.594645	1.750307
Barcos	0.410974	1.308861	-1.070031	-0.973952
Autobus	1.039543	-0.926627	-1.249022	-1.622006
Coche	-0.505677	2.598641	0.625704	0.353713

```
# Ordenamos por valores.
# En caso de empate en "a", ordena por valores en "c"
```



```
print(frame.sort_values(["a", "c"]))
```

	d	a	b	c
Autobus	1.039543	-1.622006	-1.249022	-0.926627
Barcos	0.410974	-0.973952	-1.070031	1.308861
Coche	-0.505677	0.353713	0.625704	2.598641
Trenes	-0.697110	1.750307	-0.594645	0.685423

```
#Si hubiera valores nulos se pueden poner al principio
frame.loc["Trenes", "a"] = np.nan
print(frame.sort_values(["a"], na_position="first"))
```

	d	a	b	c
Trenes	-0.697110	NaN	-0.594645	0.685423
Autobus	1.039543	-1.622006	-1.249022	-0.926627
Barcos	0.410974	-0.973952	-1.070031	1.308861
Coche	-0.505677	0.353713	0.625704	2.598641

La función **rank** de Pandas se utiliza para asignar rangos a los valores de una serie o columna en un DataFrame. Funciona ordenando los valores de menor a mayor y asignando un rango desde 1 hasta n, donde n es el número de valores no nulos. Si existen valores duplicados, la función maneja los empates según el método especificado. Por defecto, usa el método "average", que asigna a los valores repetidos el promedio de los rangos que ocuparían.

```
import pandas as pd

obj = pd.Series([7, -5, 7, 4, 2, 0, 4])

print(obj)
```

0	7
1	-5
2	7
3	4
4	2
5	0
6	4

```
dtype: int64
```

```
print(obj.rank())
```

```
0    6.5
1    1.0
2    6.5
3    4.5
4    3.0
5    2.0
6    4.5
dtype: float64
```

```
# Desempata con el orden en que aparecen los valores
print(obj.rank(method="first"))
```

```
0    6.0
1    1.0
2    7.0
3    4.0
4    3.0
5    2.0
6    5.0
dtype: float64
```

```
# Ejemplo dataframe
frame = pd.DataFrame({"a": [4.3, 7, -3, 2],
                      "b": [0, 1, 2, 0],
                      "c": [-2, 5, 8, -2.5]})
```

```
print(frame)
```

```
   a  b  c
0  4.3  0 -2.0
1  7.0  1  5.0
2 -3.0  2  8.0
3  2.0  0 -2.5
```

```
# Hace el rango por filas
print(frame.rank(axis="columns"))
```

	a	b	c
0	3.0	2.0	1.0
1	3.0	1.0	2.0
2	1.0	2.0	3.0
3	3.0	2.0	1.0

La tabla de a continuación muestra los métodos existentes de desempate con rank.

Método	Descripción
"average"	Método por defecto. Asigna el rango medio a cada entrada de los valores empatados
"min"	Utiliza el rango mínimo para el grupo empatado
"max"	Utiliza el rango máximo para el grupo empatado
"first"	Asigna rangos en el orden en que aparecen los valores en los datos
"dense"	Igual que "min". Pero el rango va aumentando al cambiar entre grupos de repetidos en vez de entre los números repetidos.

Índices de ejes con etiquetas duplicadas

Aunque muchas de las funciones de pandas requieren que las etiquetas de los índices no se repitan entre si, no es algo obligatorio. De hecho, existe la posibilidad de consultar si en un índice hay repetidos o no con la propiedad `is_unique`.

```
import pandas as pd
import numpy as np

obj = pd.Series([7, -5, 7, 4])

0    7
1   -5
2    7
3    4
dtype: int64

print(obj.index.is_unique)

True

frame = pd.DataFrame(np.random.standard_normal((3,3)),
                     index=["Repetido", "Repetido", "Diferente"])
```

	0	1	2
Repetido	0.752429	-0.802467	-0.449333
Repetido	1.311927	-1.583955	0.396837
Diferente	0.514103	-0.713375	0.522409

```
print(frame.index.is_unique)
```

False

RESUMIR Y CALCULAR ESTADÍSTICAS DESCRIPTIVAS

Los objetos de pandas están equipados con un conjunto de métodos matemáticos y estadísticos que permiten analizar y resumir datos fácilmente. La mayoría de estos métodos entran en la categoría de reducciones o estadísticas de resumen, es decir, métodos que toman un conjunto de valores y generan un único resultado (como por ejemplo la suma o el promedio).

Una característica útil es que estos métodos pueden ignorar los valores NaN (datos faltantes) para no afectar los cálculos, en lugar de devolver un resultado NaN. Esto significa que si en una columna hay algunos valores faltantes, pandas puede seguir operando con los datos disponibles sin necesidad de imputarlos antes de calcular estadísticas.

```
import pandas as pd
import numpy as np

frame = pd.DataFrame([[1.4, np.nan], [7.1, -4.5], [np.nan,
np.nan], [0.75, -1.3]],
                      index=list("abcd"),
                      columns=["uno", "dos"])

print(frame)
```

	uno	dos
a	1.40	NaN
b	7.10	-4.5
c	NaN	NaN
d	0.75	-1.3

```
# Suma los valores de las filas de cada columna
print(frame.sum(axis="index"))
```

```

        uno      9.25
        dos     -5.80
        dtype: float64

# No excluimos los valores faltantes
print(frame.sum(axis="index", skipna=False))

        uno      NaN
        dos      NaN
        dtype: float64

```

Métodos de estadística descriptivas y de resumen:

Método	Descripción
count	Número de valores que no son nulos
describe	Calcula un conjunto de estadísticas de resumen: count, mean, std, min, max y percentiles
min, max	Calcula los valores mínimo o máximo, respectivamente
argmin, argmax	Devuelve la posición del valor mínimo o máximo, respectivamente
idxmin, idxmax	Devuelve los nombres de los índices en las que están el valor mínimo o el máximo, respectivamente
quantile	Calcula el cuantil de muestra entre 0 y 1. Por defecto calcula el 0.5
sum	Suma de valores
mean	Calcula la media
median	Calcula la mediana, es decir, el cuantil 50%
mad	Desviación media absoluta del valor promedio
prod	Producto de todos los valores
var	Varianza de los valores de la muestra
std	Desviación estándar entre los valores de la muestra
skew	Asimetría (tercer momento) de los valores de la muestra. Determina qué tan simétrica es la distribución, si es positiva la cola está hacia la derecha y si es negativa, hacia la izquierda.

kurt	Curtosis (cuarto momento) de los valores de la muestra. Mide qué tan concentrados están los valores en los extremos de una distribución.
cumsum	Suma acumulada de los valores
cummin, cummax	Mínimo o máximo acumulado de los valores, respectivamente
cumprod	Producto acumulado de valores
diff	Calcula la primera diferencia aritmética. Útil para analizar tendencias
pct_change	Calcula cambios de porcentaje
cov	Calcula la covarianza, cómo varían dos variables juntas
corr	Calcula la correlación, que es la covarianza normalizada

Métodos únicos, de recuentos de valores y de pertenencia a conjuntos

Método	Descripción
isin	Devuelve un array booleano indicando si cada valor de una serie o dataframe está contenido en la secuencia de valores que se le pasa como argumento
get_indexer	Encuentra las posiciones de etiquetas en un índice y devuelve sus ubicaciones numéricas en forma de array, útil para alineación de datos y búsqueda eficiente
unique	Devuelve un array de valores únicos de una serie, manteniendo el orden en que aparecen
value_counts	Cuenta la frecuencia de cada valor único en una Serie y devuelve un resultado ordenado de mayor a menor frecuencia.

```
import pandas as pd

serie = pd.Series([3, 1, 2, 3, 2, 4, 1, 3])

conteo = serie.value_counts()
print(conteo)
```

```
3    3
1    2
2    2
4    1
Name: count, dtype: int64
```

```
print(serie.isin([2]))
```

```
0    False
1    False
2     True
3    False
4     True
5    False
6    False
7    False
dtype: bool
```

```
print(serie.unique())
```

```
[3 1 2 4]
```

```
print(conteo.index.get_indexer([3,4]))
```

```
[0 3]
```

Importación y exportación de datos

IMPORTAR DATOS DESDE CUALQUIER TIPO DE ARCHIVO.....	2
EXPORTAR DATOS.....	4
EJEMPLOS DE IMPORTACIÓN.....	5
EJEMPLOS DE EXPORTACIÓN.....	11

IMPORTAR DATOS DESDE CUALQUIER TIPO DE ARCHIVO

Cuando trabajamos con datos, destinamos una importante cantidad de tiempo a la limpieza de los datos y para ello, una de las primeras acciones a realizar es la carga de los datos para conocerlos y ver a qué nos enfrentamos. Python mediante su librería Pandas nos ofrece la funcionalidad de importar y exportar datos mediante el uso de funciones sencillas de comprender y ejecutar.

Función	Descripción
<code>read_csv</code>	Crea dataframe con los datos que lee de un archivo tipo CSV (Comma-Separated Values). El delimitador por defecto es la coma.
<code>read_fwf</code>	Lee archivos de texto con columnas de ancho fijo, sin delimitadores
<code>read_clipboard</code>	Lee datos del portapapeles
<code>read_excel</code>	Lee datos tabulares de un archivo XLS o XLSX de Excel
<code>read_hdf</code>	Lee archivos HDF5 escritos por pandas
<code>read_html</code>	Lee todas las tablas que encuentra en el documento HTML dado
<code>read_json</code>	Lee datos desde un archivo, URL o cadena de texto JSON (Javascript Object Notation)
<code>read_parquet</code>	Lee el formato de archivo binario Parquet de Apache
<code>read_sql</code>	Lee los resultados de una consulta SQL (utilizando la biblioteca SQLAlchemy)
<code>read_sql_table</code>	Lee una tabla SQL completa (utilizando la biblioteca SQLAlchemy)
<code>read_xml</code>	Lee una tabla o datos de un archivo XML
<code>read_feather</code>	Lee el formato de archivo binario Feather
<code>read_spss</code>	Lee un archivo de datos creado por SPSS (.sav)
<code>read_stata</code>	Lee conjunto de datos de un formato de archivo Stata (.dta)

Estas funciones cuentan con argumentos opcionales que permiten personalizar la lectura de datos. Algunas de las opciones más comunes para leer archivos e importar datos con Python pandas son:

Argumento	Descripción
sep o delimiter	Caracteres o expresión regular que especifican el separador de campos en cada fila. Por ejemplo <code>sep="\t"</code> indica que están separados por tabuladores.
header	Especifica el número de fila que contiene los nombres de las columnas. Por defecto es cero, pero debería ser <code>None</code> si no hay fila de encabezado.
index_col	Especifica el número o nombres de columna que se debe usar como índice de fila en el resultado
skiprows	Lista de número de líneas que se deben ignorar al leer el archivo.
na_values	Secuencia de valores que se deben identificar como valores NA. Por ejemplo, puedes utilizar <code>na_values=["N/A", ""]</code> para indicar que los valores "N/A" y el texto vacío deben interpretarse como NA.
keep_default_na	Para usar la lista predeterminada de valores NA (not available) o no. Por defecto está a <code>True</code>
dtype	Especifica el tipo de datos de cada columna. Por ejemplo, <code>dtype={"col1": int, "col2": float}</code> indica que la columna "col1" contiene enteros y la columna "col2" contiene flotantes.
usecols	especifica una lista de nombres de columna que se deben leer del archivo. Por ejemplo, puedes utilizar <code>usecols=["col1", "col3"]</code> para leer solo las columnas "col1" y "col3" del archivo.
nrows	Número de filas que se leen desde el principio del archivo (sin contar el encabezado)
skip_footer	Número de líneas que se ignoran del final del archivo.
parse_dates	Especifica una lista de nombres de columnas que se deben interpretar como fechas <i>datetime</i> .
decimal	Especifica el separador de los decimales en números. Por defecto es "." pero puede ponerse como ",",
thousands	Especifica el separador de los miles para números. Por defecto es <code>None</code> pero puede ponerse como ",", o "."

Hay muchas otras opciones disponibles, y no todas las opciones están disponibles para todas las funciones de lectura de Pandas. Para más información de cada función de lectura recordad que podéis acudir a la [documentación online](#)

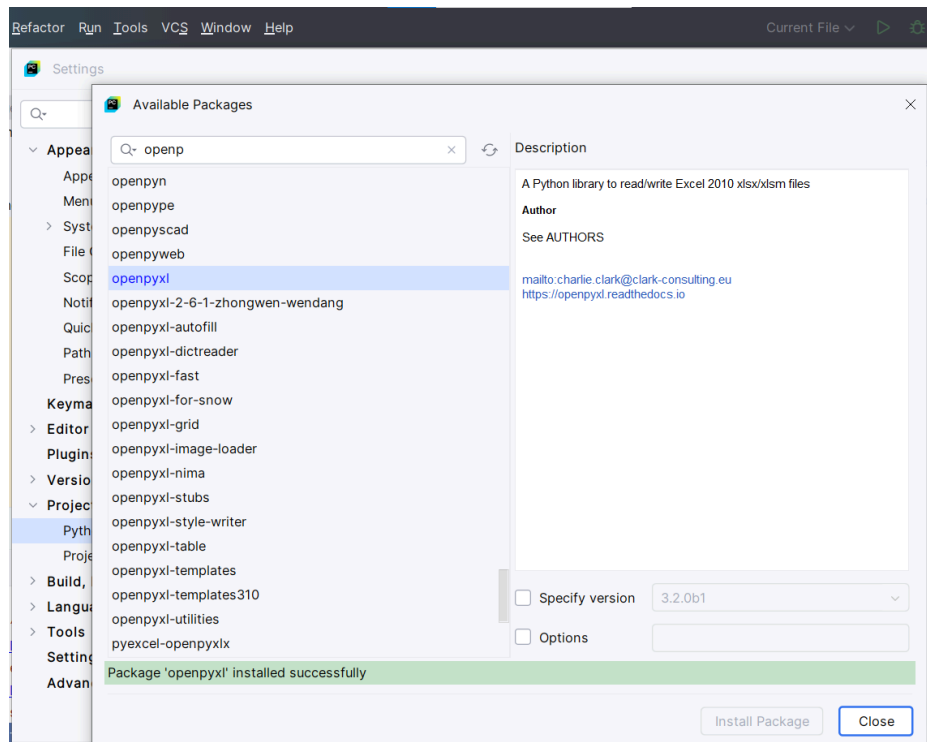
EXPORTAR DATOS

Los datos también se pueden exportar a un formato delimitado empleando los métodos `to_`*formato* del objeto DataFrame.

Método	Descripción
<code>to_csv</code>	Escribe el objeto DataFrame en un archivo separado por comas (csv).
<code>to_clipboard</code>	Copia el objetos en el portapapeles del sistema
<code>to_excel</code>	Copia el objeto en un archivo de Excel
<code>to_html</code>	Representa el DataFrame como una tabla HTML
<code>to_json</code>	Convierte el objeto en un string JSON
<code>to_parquet</code>	Escribe el DataFrame en el formato Parquet
<code>to_sql</code>	Escribe los datos del DataFrame en una base de datos SQL
<code>to_xml</code>	Representa el DataFrame en un documento XML
<code>to_feather</code>	Escribe el DataFrame en formato feather
<code>to_stata</code>	Exporta el objeto DataFrame a formato Stata dta
<code>to_latex</code>	Convierte el objeto en una tabla LaTeX tabular, longtable o anidada

EJEMPLOS DE IMPORTACIÓN

Para algunas de las funciones, tenemos que instalar algunas dependencias que se irán indicando con los ejemplos. Para trabajar con archivos excel necesitamos añadir la librería openpyxl. En Pycharm le damos a File → Settings → Project: → Python Interpreter → +. En Jupyter con `pip install nombre_libreria`.



```
import pandas as pd
```

```
empleados = pd.read_excel("Lista-de-clientes.xlsx")  
print(empleados)
```

	ID	...	Grupo de clientes
0	C0001	...	A
1	C0002	...	E
2	C0003	...	E
3	C0004	...	E
4	C0005	...	D
..	...	...	...
994	C0995	...	B
995	C0996	...	D
996	C0997	...	A
997	C0998	...	D
998	C0999	...	A

[999 rows x 9 columns]

```
portapapeles = pd.read_clipboard()
print(portapapeles)
```

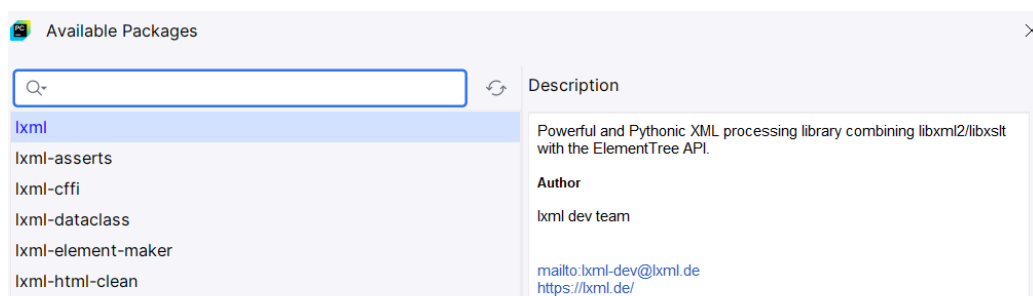
	ID	Nombre completo	Fecha de nacimiento
0	C0001	Leandra Anna Malo Alba	08/12/1984
1	C0002	Severo Granados Iglesia	12/08/1986
2	C0003	Lucho Andreu Amat	16/04/1990
3	C0004	Matias Mauricio Castillo Barrera	02/12/1996
4	C0005	Mauricio Guijarro Castelló	14/05/1984
5	C0006	Isaura Leyre Avilés Pelayo	18/04/1987
6	C0007	Soraya Morera-Lago	27/11/1990
7	C0008	Victoriano Tapia-Cabanillas	06/03/1981
8	C0009	Nidia Saez Campoy	28/04/1980
9	C0010	Teófila Villanueva Molina	04/01/1987
10	C0011	Trini de Alberdi	04/12/1990
11	C0012	Dani Baena	20/10/1977
12	C0013	Angelina de Arregui	21/02/2000
13	C0014	Samuel de Carranza	16/05/1999
14	C0015	Jacinta Montenegro Garcés	13/03/1994
15	C0016	Lisandro Delgado Nadal	18/04/1980

```
json = pd.read_json("datos.json")
print(json)
```

[2 rows x 4 columns]

	Titulo	Tiempo	Final
0	RESPONSIVE WEB DESIGN CON BOOTSTRAP	01:07:20	si
1	Presentación	00:03:20	si
2	*** Definición del proyecto de diseño	NaN	si
3	- Introducción	00:04:50	si
4	- Entorno de trabajo	00:04:50	si
5	- Esquema y estructura del proyecto de diseño	00:07:45	si
6	- Preparación de imágenes en Photoshop	00:06:20	si
7	- Prototipado de la página web	00:05:25	si
8	- Estructura HTML5	00:10:35	si
9	*** Twitter Bootstrap (Framework Responsive De...	NaN	si
10	- Introducción a bootstrap	00:06:25	si
11	- Estructura HTML5 básica para bootstrap	00:04:20	si
12	- Sistema de rejilla (grid system) fijo y fluido	00:11:14	si
13	- Capacidades Responsive (@media - CSS)	00:09:05	si
14	- Base CSS - Tipografías y elementos en línea	00:11:49	si

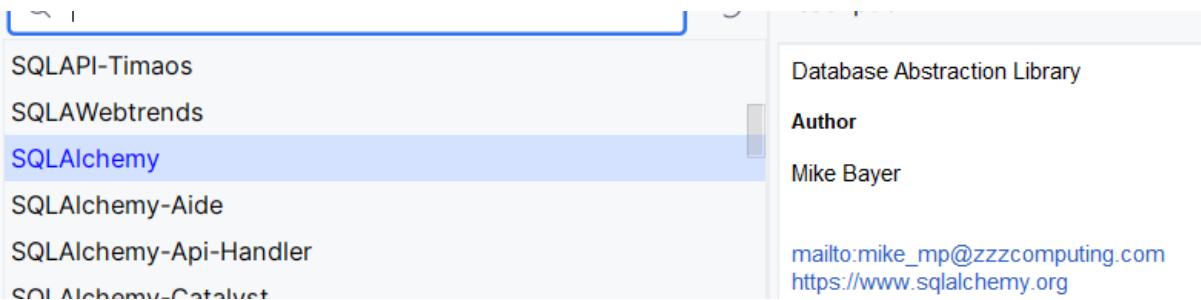
Para trabajar con archivos xml y html necesitamos añadir la librería lxml



```
html = pd.read_html("simple-table.html")
print(html)

[          0  ...          3
0  Hi, I'm your first cell.  ...  I'm your fourth cell.
1  Second row, first cell.  ...          Cell 4.]
```

Para interactuar con bases de datos SQL necesitamos añadir la librería SQLAlchemy



```
import pandas as pd
import sqlite3
import sqlalchemy

#Nos conectamos a la base de datos
con = sqlite3.connect('Our DataBase')
cur = con.cursor()

#Creamos la tabla
cur.execute(''' CREATE TABLE IF NOT EXISTS COURSES
                ([course_id] INTEGER PRIMARY KEY,
                 [course_name] TEXT,
                 [fee] INTEGER,
                 [duration] TEXT,
                 [discount] INTEGER)''')

#Insertamos filas en la tabla
cur.execute('''INSERT INTO COURSES (course_id, course_name, fee, duration, discount)
                VALUES (1,'Spark',25000,'50 Days', 2000),
                (2,'Pandas',20000,'35 Days', 1000),
                (3,'Java',15000,'35 Days', 800),
                (4,'Python',15000,'30 Days', 500),
                (5,'PHP',28000,'30 Days', 800)''')

#Leemos la tabla
sql_query = pd.read_sql('SELECT * FROM COURSES', con)
```

```
print(sql_query)
```

	course_id	course_name	fee	duration	discount
0	1	Spark	25000	50 Days	2000
1	2	Pandas	20000	35 Days	1000
2	3	Java	15000	35 Days	800
3	4	Python	15000	30 Days	500
4	5	PHP	28000	30 Days	800

En el ejemplo de csv vamos a emplear algunos de los argumentos opcionales para ver como funcionarían

```
import pandas as pd
```

```
# Usamos distintos argumentos opcionales
# Especificamos el separador y definimos la fila con los encabezados
lectural1 = pd.read_csv("InventoryLot_v4.csv", sep=";", header=0)
```

```
# Mostramos solo las primeras 5 filas
print(lectural1.head())
```

	product	partNumber	...	storageDate
0	PS-SL-A287	...	2022-01-01T10:00:00	
1	PS-SL-A288	...	2022-01-01T00:00:00	
2	PS-SL-H309	...	2022-01-01T09:00:00	
3	PS-SL-H310	...	2022-01-01T00:30:00	
4	PS-SL-F342	...	2022-01-01T00:00:00	

[5 rows x 15 columns]

```
# Definimos la columna de índices y solo cargamos 3 columnas del fichero
lectura2 = pd.read_csv("InventoryLot_v4.csv", index_col="lotCode",
                        usecols=["lotCode", "quantity", "value"])
```

```
print(lectura2)
```

	quantity	value
lotCode		
521033	25	31250.0
3282509	20	25000.0
862421	3	3750.0
9282139	35	43750.0
240235	43	53750.0
...	...	...
8774537	118	151866.0
9695355	323	470288.0
8012120	273	227409.0
3516489	259	267288.0
4216598	27	26352.0

[154 rows x 2 columns]

```
# Omitimos las primeras 100 líneas y lee solo las siguientes 10
lectura3 = pd.read_csv("InventoryLot_v4.csv", skiprows=100, nrows=10)
print(lectura3)
```

```

      7900014  LT-1  ITEM  387  ...    7353  USD  https://foo.com  2022-01-01T00:00:00
0  7900015  LT-1  ITEM  155  ...     465  USD  https://foo.com  2022-01-01T00:00:00
1  9664144  LT-1  ITEM   37  ...     888  USD  https://foo.com  2022-01-01T00:00:00
2  9665306  LT-1  ITEM  233  ...     699  USD  https://foo.com  2022-01-01T00:00:00
3  2530003  LT-1  ITEM  214  ...  276060  USD  https://foo.com  2022-01-01T00:00:00
4  2530020  LT-1  ITEM  455  ...  536900  USD  https://foo.com  2022-01-01T00:00:00
5  7903986  LT-1  ITEM  228  ...  225720  USD  https://foo.com  2022-01-01T00:00:00
6  8220072  LT-1  ITEM   44  ...   63888  USD  https://foo.com  2022-01-01T00:00:00
7  83008200 LT-1  ITEM  116  ...  278168  USD  https://foo.com  2022-01-01T00:00:00
8  83008300 LT-1  ITEM  109  ...   500528  USD  https://foo.com  2022-01-01T00:00:00
9  83008400 LT-1  ITEM  177  ...   978810  USD  https://foo.com  2022-01-01T00:00:00

```

```
[10 rows x 15 columns]
```

```
# Especificamos que reconozca como valores nulos "N/A", "Desconocido" y vacío.
```

```
# No considera los valores predeterminados de pandas
```

```
lectura4 = pd.read_csv("InventoryLot_v4.csv", na_values=["N/A",
                                                         "Desconocido", ""], keep_default_na=False)
print(lectura4.info())
```

```
[10 rows x 15 columns]
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 154 entries, 0 to 153
```

```
Data columns (total 15 columns):
```

#	Column	Non-Null Count	Dtype
0	product.partNumber	154 non-null	object
1	location.locationIdentifier	154 non-null	object
2	inventoryType	154 non-null	object
3	quantity	154 non-null	int64
4	quantityUnits	154 non-null	object
5	expirationDate	154 non-null	object
6	inventoryParentType	152 non-null	object
7	class	153 non-null	object
8	segment	151 non-null	object
9	lotCode	154 non-null	int64
10	status	154 non-null	object
11	value	152 non-null	float64
12	valueCurrency	154 non-null	object
13	sourceLink	154 non-null	object
14	storageDate	154 non-null	object

```
dtypes: float64(1), int64(2), object(12)
```

```
memory usage: 18.2+ KB
```

```
None
```



```
# Convertimos la columna expirationDate a formato de fecha
```

```
lectura5 = pd.read_csv("InventoryLot_v4.csv",  
                        parse_dates=["expirationDate"])
```

```
print(lectura5["expirationDate"])
```

```
0      2022-12-31 00:00:00
```

```
1      2022-12-31 09:00:00
```

```
2      2022-12-31 10:00:00
```

```
3      2022-12-31 00:00:00
```

```
4      2022-12-31 05:00:00
```

```
...
```

```
149    2022-12-31 00:00:00
```

```
150    2022-12-31 00:00:00
```

```
151    2022-12-31 00:00:00
```

```
152    2022-12-31 00:00:00
```

```
153    2022-12-31 00:00:00
```

```
Name: expirationDate, Length: 154, dtype: datetime64[ns]
```

EJEMPLOS DE EXPORTACIÓN

```
import pandas as pd

json = pd.read_json("datos.json")
# Exportamos solo las primeras 5 filas
# "records" es el formato que le damos al string json
json.head(5).to_json("exportado.json", orient="records", indent=4)

lectural = pd.read_csv("InventoryLot_v4.csv", sep=";", header=0)

# Exportar todo el dataframe
lectural.to_csv("exportado.csv")

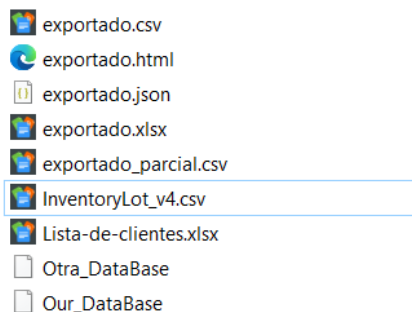
#Exportar algunas columnas y sin que se agregue una columna de índice
lectural[["product.partNumber", "quantity",
          "expirationDate"]].to_csv("exportado_parcial.csv",
                                   index=False)

html = pd.read_html("simple-table.html")
# Exportamos el primer Dataframe (tenemos que especificarlo)
html[0].to_html("exportado.html")

empleados = pd.read_excel("Lista-de-clientes.xlsx")
# Exportamos solo algunas columnas
empleados[["ID", "Nombre completo",
           "Grupo de clientes"]].to_excel("exportado.xlsx",
                                          index=False)

empleados.to_clipboard() #puedes pegarlo en un fichero de texto

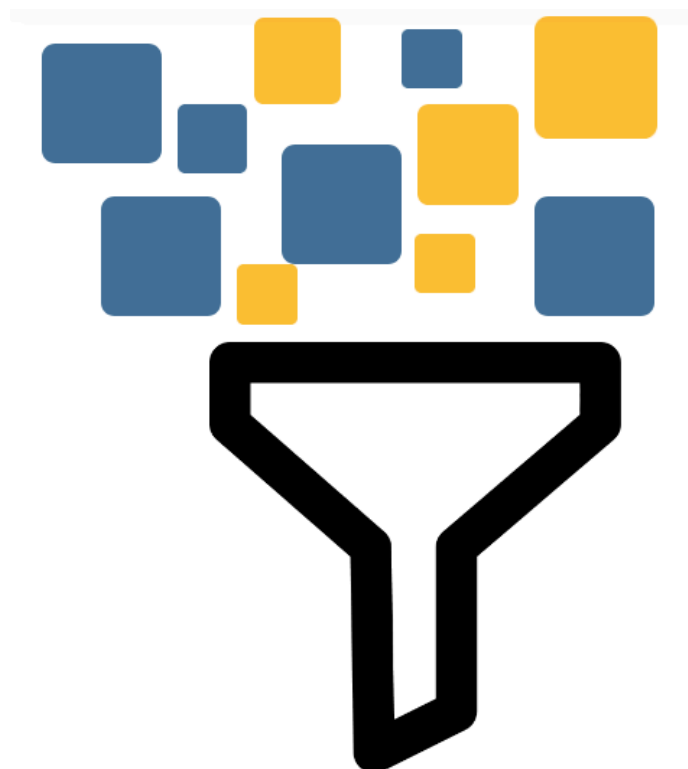
con = sqlite3.connect("Otra_DataBase")
# Exportamos el dataframe a SQLite
empleados.to_sql("INVENTORY", con, if_exists="replace",
                index=False)
con.close()
```



Se generarán los distintos ficheros.

Limpieza y preparación de los datos

Introducción.....	2
Técnicas de limpieza y preparación de datos.....	2
Gestión de los datos que faltan.....	2
Filtrado de datos ausentes.....	2
Rellenado de datos ausentes.....	4
Transformación de datos.....	5
Eliminación de duplicados.....	5
Transformación de datos mediante una función o asignación.....	7
Reemplazar valores.....	8
Discretización.....	8
Detección y filtrado de valores atípicos.....	10
Permutación y muestreo aleatorio.....	11
Agrupación de datos.....	12
Datos tipo datetime.....	13
Apartado de ampliación.....	15
Renombrar índices de eje.....	15
Calcular variables dummy o indicadoras.....	16
Manipulación de cadenas de texto.....	18
Datos categóricos.....	22



Introducción

En el análisis de datos, se emplea una considerable cantidad de tiempo en la preparación de los mismos: carga, limpieza, transformación y reordenación. Contar con información limpia y bien estructurada es esencial para obtener resultados precisos y confiables. Esta fase implica detectar y corregir errores, manejar valores nulos, eliminar duplicados, ajustar formatos y estandarizar la información para garantizar su coherencia.

La limpieza de datos es fundamental para conseguir:

- **Precisión en el Análisis:** Los datos sucios, como los datos incompletos, duplicados o erróneos, pueden distorsionar los resultados del análisis. La limpieza de datos asegura que los resultados sean precisos y fiables.
- **Eficiencia:** Datos bien preparados permiten un análisis más rápido y eficiente. Los analistas pueden centrarse en el análisis propiamente dicho en lugar de lidiar con problemas de calidad de datos.
- **Toma de decisiones informadas:** Decisiones basadas en datos precisos y relevantes mejora la precisión de las decisiones.
- **Cumplimiento y normativas:** La limpieza de datos ayuda a cumplir con normativas y estándares de calidad de datos, evitando posibles sanciones y mejorando la reputación de la empresa.

Técnicas de limpieza y preparación de datos

Gestión de los datos que faltan

En muchas aplicaciones de datos suele haber datos ausentes, faltantes o perdidos. Uno de los objetivos de pandas es que el trabajo con este tipo de datos sea lo más sencillo posible. Algo que se suele hacer es identificar estos valores y eliminarlos o usar técnicas para reemplazarlos. Aunque en apuntes anteriores ya vimos métodos para trabajar con valores nulos, vamos a repasarlos.

Filtrado de datos ausentes

Función	Descripción
dropna	Elimina filas o columnas que contienen valores nulos
isna	Devuelve booleanos indicando si el valor es nulo/ausente
notna	Devuelve booleanos al contrario que isna. Consulta que no sean valores nulos/ausentes

```
import pandas as pd
import numpy as np
```

```
prueba = pd.DataFrame({
    "A": [1, 2, np.nan, 4],
    "B": [5, np.nan, np.nan, 8],
    "C": [9, 10, np.nan, 12]
})
```

	A	B	C
0	1.0	5.0	9.0
1	2.0	NaN	10.0
2	NaN	NaN	NaN
3	4.0	8.0	12.0

```
# Elimina las filas que contengan algún valor nulo
prueba1 = prueba.dropna()
```

	A	B	C
0	1.0	5.0	9.0
3	4.0	8.0	12.0

```
# Elimina las columnas que contengan algún valor nulo
prueba2 = prueba.dropna(axis="columns")
```

```
Empty DataFrame
Columns: []
Index: [0, 1, 2, 3]
```

```
# Solo elimina la fila/columna con todos los valores nulos
prueba3 = prueba.dropna(how="all")
```

	A	B	C
0	1.0	5.0	9.0
1	2.0	NaN	10.0
3	4.0	8.0	12.0

```
# Elimina las filas que contengan más de 2 valores faltantes
prueba4 = prueba.dropna(thresh=2)
```

	A	B	C
0	1.0	5.0	9.0
1	2.0	NaN	10.0
3	4.0	8.0	12.0

Rellenado de datos ausentes

Para la mayoría de los casos se debe emplear el método *fillna*

Argumentos función fillna	Descripción
value	Valor escalar u objeto de tipo diccionario que se utiliza para rellenar valores faltantes
axis	Eje que rellenar. Filas (0 ó "index") o columnas (1 ó "columns")
limit	Determina cuántos valores NaN se pueden rellenar consecutivamente en cada columna o fila antes de detenerse.

```
import pandas as pd
import numpy as np

original = pd.DataFrame(np.random.standard_normal((7,3)))

# Ponemos nulos los valores de las 4 primeras filas de la columna 1
original.iloc[:4,1] = np.nan
# Ponemos nulos los valores de las 2 primeras filas de la columna 2
original.iloc[:2,2] = np.nan
```

	0	1	2
0	0.062692	NaN	NaN
1	-1.379858	NaN	NaN
2	1.934856	NaN	0.930063
3	-0.374948	NaN	0.557342
4	-0.532310	-1.088679	0.649480
5	0.781064	1.857998	0.300983
6	0.031965	-0.518349	-0.973399

```
# Rellenamos los huecos con ceros
rellenado1 = original.fillna(0)
```

	0	1	2
0	0.632250	0.000000	0.000000
1	-1.216332	0.000000	0.000000
2	-1.092250	0.000000	0.878229
3	-0.820047	0.000000	-0.438044
4	1.934321	0.583819	-1.647841
5	0.112707	0.032173	0.855852
6	-1.281958	-1.872129	0.020458

Rellenamos los huecos con diccionario

```
rellenado2 = original.fillna({1: 1.5, 2: 2.5})
```

	0	1	2
0	0.436319	1.500000	2.500000
1	0.293295	1.500000	2.500000
2	0.577954	1.500000	-0.131986
3	-0.780675	1.500000	-1.093722
4	0.473630	-0.127320	1.448386
5	-1.122823	-0.446331	-0.668205
6	0.612272	-1.348449	1.522857

Rellenamos los huecos con interpolación

Ponemos que se rellenen dos huecos máximo

```
rellenado3 = original.bfill(limit=2)
```

	0	1	2
0	-1.516331	NaN	0.669728
1	-0.471286	NaN	0.669728
2	-0.490355	1.733353	0.669728
3	1.117432	1.733353	0.218140
4	-1.586337	1.733353	-0.181955
5	0.596220	-0.212539	-0.713754
6	0.044582	-0.898197	1.243516

Rellenamos los huecos con la media

```
rellenado4 = original.fillna(original.mean())
```

	0	1	2
0	-0.251266	0.703885	-0.107652
1	1.873014	0.703885	-0.107652
2	1.663624	0.703885	-1.679340
3	2.073861	0.703885	-0.273092
4	-0.037209	0.049622	0.239458
5	-1.047683	0.978342	0.114876
6	1.412373	1.083691	1.059837

Transformación de datos

Eliminación de duplicados

La eliminación de duplicados permite evitar redundancias que pueden distorsionar el análisis, asegurando que cada observación sea única y representativa. Existe el método `uplicated()`, que indica si alguna fila es un duplicado devolviendo un `True`, y el `drop_duplicates()`, que elimina registros repetidos de manera eficiente.

```
import pandas as pd

df = pd.DataFrame({
    "A": [1, 2, 3, 1, 2, 3],
    "B": ["X", "Y", "Z", "X", "Y", "Z"],
    "C": [10, 20, 30, 10, 20, 30],
    "D": [100, 200, 300, 100, 200, 300]
})

      A  B   C   D
0  1  X  10  100
1  2  Y  20  200
2  3  Z  30  300
3  1  X  10  100
4  2  Y  20  200
5  4  Z  30  300

# Detectamos duplicados
print(df.duplicated())

      0    False
      1    False
      2    False
      3     True
      4     True
      5    False
      dtype: bool

# Eliminamos filas duplicadas
sin_repes = df.drop_duplicates()

      A  B   C   D
0  1  X  10  100
1  2  Y  20  200
2  3  Z  30  300
5  4  Z  30  300
```



```
# Eliminamos duplicados tomando de referencia una
columna
sin_repes2 = df.drop_duplicates(subset=["B"])
```

	A	B	C	D
0	1	X	10	100
1	2	Y	20	200
2	3	Z	30	300

Transformación de datos mediante una función o asignación

La función `map()` permite aplicar otra función definida previamente a cada elemento de una lista iterable. Es una forma conveniente de realizar transformaciones elemento a elemento y otras operaciones de datos relacionadas con su limpieza.

```
import pandas as pd

aquarium = pd.DataFrame({
    "Salas": [1, 2, 3, 4, 5],
    "Especie": ["Boquerón", "Trucha", "Barbo", "Sardina", "Carpa"],
    "Total peces": [10, 8, 15, 20, 9],
})
```

	Salas	Especie	Total peces
0	1	Boquerón	10
1	2	Trucha	8
2	3	Barbo	15
3	4	Sardina	20
4	5	Carpa	9

```
pez_agua = {
    "Boquerón": "agua salada",
    "Trucha": "agua dulce",
    "Lubina": "agua salada",
    "Dorada": "agua salada",
    "Barbo": "agua dulce",
    "Sardina": "agua salada",
    "Perca": "agua dulce",
    "Carpa": "agua dulce"
}
```

```
# Añadimos columna que indica el tipo de agua del pez
def get_habitat(n):
    return pez_agua[n]
```

```
# Añadimos columna con el tipo de agua
aquarium["Hábitat"] = aquarium["Especie"].map(get_habitat)
```

	Salas	Especie	Total peces	Hábitat
0	1	Boquerón	10	agua salada
1	2	Trucha	8	agua dulce
2	3	Barbo	15	agua dulce
3	4	Sardina	20	agua salada
4	5	Carpa	9	agua dulce

Reemplazar valores

La función *replace()* ofrece otra forma sencilla y flexible de modificar valores.

```
import pandas as pd
import numpy as np
```

```
numeros = pd.Series([1, 3, -999, 0, 2, -1000, -999, 5, 9])
```

```
0      1
1      3
2    -999
3       0
4       2
5   -1000
6    -999
7        5
8         9
dtype: int64
```

```
# -999 y -1000 podrían ser valores centinelas para datos faltantes
# Los reemplazamos para poder tratarlos mejor con pandas
```

```
numeros= numeros.replace([-999,-1000],np.nan)
```

```
0      1.0
1      3.0
2      NaN
3      0.0
4      2.0
5      NaN
6      NaN
7      5.0
8      9.0
dtype: float64
```

Discretización

La discretización es el proceso de convertir datos continuos en categóricos, dividiéndolos en intervalos o grupos. Pandas ofrece dos funciones:

- `cut()`: crea intervalos de tamaño fijo o definidos manualmente
- `qcut()`: que crea intervalos con el mismo número de elementos en cada uno (cuantiles).

```
import pandas as pd
'''
    Imaginemos que tenemos datos sobre un grupo de personas
    y queremos agruparlos por edad
    Lo queremos dividir en tres intervalos:
    De 18 a 25
    De 26 a 40
    De 41 a 60
'''
edad = [20, 49, 29, 28, 31, 26, 47, 32]
grupos = [18, 25, 40, 60]

# Vamos a ponerle nombre a cada grupo
division = pd.cut(edad, grupos,
                  labels=["Joven", "Adulto Joven", "Adulto"])

print(division)

['Joven', 'Adulto', 'Adulto Joven', 'Adulto Joven', 'Adulto Joven', 'Adulto Joven', 'Adulto', 'Adulto Joven']
Categories (3, object): ['Joven' < 'Adulto Joven' < 'Adulto']

#Muestra a qué intervalo pertenece cada elemento de "edad"
print(division.codes)

      [0 2 1 1 1 1 2 1]

#Muestra cuantos elementos hay en cada intervalo
print(division.value_counts())

      Joven      1
    Adulto Joven  5
      Adulto      2
'''
    Ahora creamos 4 grupos con la misma cantidad de elementos
    precision=2 limita la precisión decimal a 2 dígitos
'''
iguales = pd.qcut(edad, 4, precision=2)
print(iguales)
```

```

[(19.99, 27.5], (35.75, 49.0], (27.5, 30.0], (27.5, 30.0], (30.0, 35.75], (19.99, 27.5], (35.75, 49.0], (30.0, 35.75]]
Categories (4, interval[float64, right]): [(19.99, 27.5] < (27.5, 30.0] < (30.0, 35.75] <
                                           (35.75, 49.0]]

print(iguales.value_counts())

           (19.99, 27.5]    2
           (27.5, 30.0]    2
           (30.0, 35.75]    2
           (35.75, 49.0]    2
Name: count, dtype: int64

```

Detección y filtrado de valores atípicos

Los valores atípicos (outliers) son datos que se alejan significativamente del resto de los valores en un conjunto de datos. Pueden ser errores, datos extremos o información relevante. El método más común para detectarlos es el [rango intercuartílico](#).

```

import pandas as pd
import numpy as np

# Creamos DataFrame con valores atípicos
df = pd.DataFrame({"Ventas": [100, 120, 110, 105, 5000, 130,
                              140, 9000, 150, 160]})

           Ventas
0           100
1           120
2           110
3           105
4          5000
5           130
6           140
7          9000
8           150
9           160

# Calculamos cuartiles y la diferencia
Q1 = df["Ventas"].quantile(0.25)
Q3 = df["Ventas"].quantile(0.75)
IQR = Q3 - Q1

# Calculamos límites para que un valor se considere atípico
limite_inferior = Q1 - 1.5 * IQR

```

```

limite_superior = Q3 + 1.5 * IQR

# Detectamos valores atípicos
outliers = df[(df["Ventas"] < limite_inferior) |
               (df["Ventas"] > limite_superior)]
print(outliers)

      Ventas
4      5000
7      9000

# Filtramos el dataFrame
df_filtrado = df[(df["Ventas"] >= limite_inferior) &
                  (df["Ventas"] <= limite_superior)]
print(df_filtrado)

      Ventas
0      100
1      120
2      110
3      105
5      130
6      140
8      150
9      160

```

Permutación y muestreo aleatorio

Son técnicas clave en el análisis de datos, especialmente cuando queremos hacer análisis exploratorio, reducir el tamaño de un dataset o validar modelos de Machine Learning. Consiste en reorganizar los datos de forma aleatoria para poder sacar una muestra. Se puede hacer de distintas formas.

```

import pandas as pd
import numpy as np

# Creamos DataFrame con valores del 0 al 34
df = pd.DataFrame(np.arange(35).reshape(5, 7))
print(df)

```

```

      0  1  2  3  4  5  6
0  0  1  2  3  4  5  6
1  7  8  9 10 11 12 13
2 14 15 16 17 18 19 20
3 21 22 23 24 25 26 27
4 28 29 30 31 32 33 34

# Subconjunto aleatorio de 3 filas
print(df.sample(n=3))

      0  1  2  3  4  5  6
3 21 22 23 24 25 26 27
4 28 29 30 31 32 33 34
1  7  8  9 10 11 12 13

# Elegimos el 50% de las columnas
print(df.sample(frac=0.5, axis=1))

      1  2  0  6
0  1  2  0  6
1  8  9  7 13
2 15 16 14 20
3 22 23 21 27
4 29 30 28 34

# Con repeticiones de filas habilitadas
print(df.sample(n=8, replace=True))

      0  1  2  3  4  5  6
2 14 15 16 17 18 19 20
4 28 29 30 31 32 33 34
2 14 15 16 17 18 19 20
2 14 15 16 17 18 19 20
2 14 15 16 17 18 19 20
0  0  1  2  3  4  5  6
4 28 29 30 31 32 33 34
4 28 29 30 31 32 33 34

```

Agrupación de datos

El método **groupby()** de Pandas es una herramienta fundamental para agrupar datos de un DataFrame según una o varias columnas. Es una de las funciones más poderosas y útiles, especialmente cuando toca trabajar con grandes volúmenes de datos que deben ser segmentados en grupos significativos para el análisis.

Vamos a ver un ejemplo de su uso: Supongamos que tenemos un DataFrame con datos de ventas de productos y queremos obtener el total de ventas por categoría:

```
import pandas as pd

df = pd.DataFrame({
    'Producto': ['A', 'B', 'A', 'B', 'C'],
    'Categoria': ['Ropa', 'Ropa', 'Electrónica', 'Electrónica',
'Ropa'],
    'Ventas': [100, 200, 150, 300, 120]
})

# Agrupamos por 'Categoria' y sumamos las ventas
ventas_por_categoria = df.groupby('Categoria')['Ventas'].sum()

print(ventas_por_categoria)
```

Categoria	
Electrónica	450
Ropa	420

Name: Ventas, dtype: int64

Este método es común verlo acompañado de funciones como `sum()`, `mean()`, `count()`, `min()`, `max()`... Se pueden aplicar varias usando `agg()`.

```
df.groupby('Categoria')['Ventas'].agg(['sum', 'mean', 'max'])
```

También permite aplicar funciones propias sobre cada grupo utilizando `apply()` o filtrar grupos que cumplan una condición específica usando `filter()`.

Datos tipo *datetime*

En Pandas, *datetime* es un tipo de dato crucial para el trabajo con series temporales, fechas y horas. Pandas ofrece la función **`pd.to_datetime()`** para convertir cadenas de texto o números en objetos de fecha y hora *datetime*. Una vez convertidos, puedes realizar operaciones como filtrar por rango de fechas, extraer años o meses o calcular diferencias entre fechas.

```
import pandas as pd
```

```
df = pd.DataFrame({
    'Fecha_Venta': ['2021-01-01', '2021-02-01', '2021-03-01'],
    'Ventas': [200, 250, 300]
})
```

```
print(df["Fecha_Venta"]) #Vemos que NO es tipo datetime
```

```
0    2021-01-01
1    2021-02-01
2    2021-03-01
Name: Fecha_Venta, dtype: object
```

```
# Convertimos a tipo datetime
df['Fecha_Venta'] = pd.to_datetime(df['Fecha_Venta'])
print(df["Fecha_Venta"])
```

```
0    2021-01-01
1    2021-02-01
2    2021-03-01
Name: Fecha_Venta, dtype: datetime64[ns]
```

```
# Esto nos permite filtrar por ejemplo por mes
df_filtrado = df[df['Fecha_Venta'].dt.month == 2]
print(df_filtrado)
```

```
      Fecha_Venta  Ventas
1  2021-02-01      250
```


Apartado de ampliación.

Lo que aparece de aquí en adelante no entra en el examen.

Renombrar índices de eje

Igual que los valores de una serie, las etiquetas de eje se pueden cambiar. Renombrar ejes en un DataFrame de Pandas permite mejorar la claridad y comprensión de los datos, facilitando su análisis e interpretación. En muchas ocasiones, los nombres de filas o columnas pueden ser genéricos, poco descriptivos o confusos, por lo que es útil ajustarlos para que reflejen mejor el contenido de la información.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({'Date': ['1/1/2021', '1/1/2021', '2/1/2021',
                           '2/1/2021', '1/1/2021', '1/1/2021', '2/1/2021',
                           '2/1/2021'],
                  'City': ['Londres', 'Madrid', 'Tokio',
                           'Liverpool',
                           'Berlín', 'Zafra', 'Badajoz', 'Cáceres'],
                  'Temperature': [25, 29, 28, 31, 26, 27, 22, 32],
                  'Humidity': [46, 50, 52, 59, 42, 45, 46, 43]})
```

	Date	City	Temperature	Humidity
0	1/1/2021	Londres	25	46
1	1/1/2021	Madrid	29	50
2	2/1/2021	Tokio	28	52
3	2/1/2021	Liverpool	31	59
4	1/1/2021	Berlín	26	42
5	1/1/2021	Zafra	27	45
6	2/1/2021	Badajoz	22	46
7	2/1/2021	Cáceres	32	43

```
# Cambiamos filas y columnas con rename
# Pero crea objeto nuevo, no modifica el original
renombrado = df.rename(index={0:"primera"},
                      columns={"Date": "Fecha",
                              "City": "Ciudad",
                              "Temperature": "Temperatura",
                              "Humidity": "Humedad"}
```

```

    })

print(renombrado)


```

	Fecha	Ciudad	Temperatura	Humedad
primera	1/1/2021	Londres	25	46
1	1/1/2021	Madrid	29	50
2	2/1/2021	Tokio	28	52
3	2/1/2021	Liverpool	31	59
4	1/1/2021	Berlín	26	42
5	1/1/2021	Zafra	27	45
6	2/1/2021	Badajoz	22	46
7	2/1/2021	Cáceres	32	43

```

# Podemos cambiar las filas también podemos con map
# En este caso sí se modifica el original
nuevo_indice = {
    "primera": "Enero_1",
    1: "Enero_2",
    2: "Febrero_1",
    3: "Febrero_2",
    4: "Enero_3",
    5: "Enero_4",
    6: "Febrero_3",
    7: "Febrero_4"
}

def indice_mes(x):
    return nuevo_indice[x]

renombrado.index = renombrado.index.map(nuevo_indice)


```

	Fecha	Ciudad	Temperatura	Humedad
Enero_1	1/1/2021	Londres	25	46
Enero_2	1/1/2021	Madrid	29	50
Febrero_1	2/1/2021	Tokio	28	52
Febrero_2	2/1/2021	Liverpool	31	59
Enero_3	1/1/2021	Berlín	26	42
Enero_4	1/1/2021	Zafra	27	45
Febrero_3	2/1/2021	Badajoz	22	46
Febrero_4	2/1/2021	Cáceres	32	43

Calcular variables dummy o indicadoras

Una variable dummy es aquella que toma el valor 1 o 0 para indicar la presencia o ausencia de una cierta característica o condición. Se utiliza comúnmente en análisis estadísticos y modelos de regresión para incluir información categórica en un modelo matemático. Al cambiar los datos categóricos, que son difíciles de entender para los algoritmos, a un formato numérico nos permite agrupar los datos categóricos sin perder ninguna información.

```
import pandas as pd
import numpy as np

#Creamos dataframe con datos aleatorios
df = pd.DataFrame(np.random.rand(10, 2).round(3) * 10,
                  columns=['Precio', 'Cantidad'])
#Añadimos columna con cadenas de texto que tengamos que convertir a dummy
df['Ciudad'] = np.random.choice(['Chicago', 'Boston', 'Nueva York'],
                                size=df.shape[0])
print(df)
```

	Precio	Cantidad	Ciudad
0	5.95	1.98	Boston
1	3.31	4.73	Boston
2	8.14	7.14	Nueva York
3	8.43	6.64	Boston
4	6.90	0.17	Nueva York
5	8.29	1.38	Chicago
6	5.30	7.43	Chicago
7	0.82	2.67	Nueva York
8	6.11	0.90	Boston
9	6.42	5.83	Nueva York

```
#Creamos la variable dummy
dummies = pd.get_dummies(df['Ciudad']).astype(int)

#Quitamos la columna original y metemos la dummy
df = df.drop('Ciudad', axis=1)
df = df.join(dummies)
print(df)
```

	Precio	Cantidad	Boston	Chicago	Nueva York
0	5.95	1.98	1	0	0
1	3.31	4.73	1	0	0
2	8.14	7.14	0	0	1
3	8.43	6.64	1	0	0
4	6.90	0.17	0	0	1
5	8.29	1.38	0	1	0
6	5.30	7.43	0	1	0
7	0.82	2.67	0	0	1
8	6.11	0.90	1	0	0
9	6.42	5.83	0	0	1

Manipulación de cadenas de texto

Como programadores, es común que necesitemos trabajar con cadenas de texto o strings en nuestro código. Python facilita la manipulación de strings con una gran cantidad de herramientas y funciones.

En primer lugar vamos a ver los métodos de cadenas de texto internos de Python.

Método	Descripción
count	Cuenta las veces que aparece en una cadena de texto la subcadena que especificamos como argumento
endswith	Devuelve True si la cadena de texto termina con la cadena que especificamos
startswith	Devuelve True si la cadena de texto empieza con la cadena que especificamos
join	Une los elementos de una secuencia con el string que llama al método como separador.
index/find	Devuelve la posición de la primera aparición de la subcadena de texto en la cadena donde se busca
rfind	Devuelve la posición de la última aparición de la subcadena de texto en la cadena donde se busca
replace	Sustituye las apariciones de la cadena de texto por otra cadena de texto
strip,rstrip,lstrip	Quita espacios en blanco incluyendo nuevas líneas en ambos lados, en el lado derecho o en el lado izquierdo

split	Divide la cadena de texto en listas de subcadenas utilizando el delimitador que se especifica
lower	Pasa todos los caracteres alfabéticos a minúsculas
upper	Pasa todos los caracteres alfabéticos a mayúsculas
ljust, rjust	Justifica a la izquierda o a la derecha, respectivamente, y rellena con espacios o un carácter especificado hasta que la cadena alcanza una longitud dada.

Vamos a ver algunos ejemplos de uso de estos métodos. En los comentarios aparece la salida que se produciría para evitar tener que poner tantas capturas de pantalla al tratarse de cadenas de texto.

```

texto = "Hola Mundo"

print(texto.lower())    # hola mundo
print(texto.upper())    # HOLA MUNDO

print(texto.startswith("Hola"))    # True
print(texto.endswith("Mundo"))    # True

print(texto.count("o"))    # 2 (cuántas veces aparece 'o')
print(texto.find("Mundo"))    # 5 (posición de "Mundo")
print(texto.rfind("o"))    # 9 (última aparición de 'o')

print(texto.ljust(15, "-"))    # "Hola Mundo----"
print(texto.rjust(15, "-"))    # "----Hola Mundo"

lista = ["Python", "es", "genial"]
print(" ".join(lista))    # "Python es genial"
print(texto.split())    # ['Hola', 'Mundo']

print(texto.replace("Mundo", "Python"))    # Hola Python

```

A la hora de buscar en un texto patrones de cadenas de texto o coincidencias, se utilizan las expresiones regulares (a menudo abreviadas como regex). Python usa el módulo `re` para trabajar con ellas. En Python, las expresiones regulares están soportadas por el módulo `re` que tenemos que importar. En la siguiente tabla aparecen métodos de expresiones regulares:

Método	Descripción
findall	Devuelve todos los patrones de coincidencia de un texto
match	Devuelve un objeto coincidente si el texto coincide con el patrón del principio. En caso contrario, devuelve None.
search	Recorre la cadena/secuencia dada, buscando la primera posición en la que se produzca una coincidencia con el patrón
split	Divide las cadenas donde coincida el patrón y devuelve una lista.
sub/subn	Reemplaza todas o las n primeras apariciones del patrón en la cadena de texto con la expresión de reemplazo

Caracteres especiales en expresiones regulares

Símbolo	Significado
.	Cualquier caracter
\d	Dígitos (0-9)
\w	Letras, números o guión bajo
\s	Espacio en blanco
+	Uno o más
*	Cero o más
?	Cero o uno
{n,m}	Repetición entre n y m veces
^	Inicio de línea
\$	Fin de línea

Para evitar que algunos símbolos (por ejemplo los que empiezan por \) se malinterpreten como comandos especiales, usaremos r" "

```
import re

texto = "Mi número es 654-789-123"
patron = r"\d{3}-\d{3}-\d{3}"
# Busca un número en formato XXX-XXX-XXX
```

```
match = re.search(patron, texto)
if match:
    print("Número encontrado:", match.group()) # 654-789-123

texto = "Los valores son 20, 30 y 50"
patron = r"\d+"
# Busca uno o más dígitos
numeros = re.findall(patron, texto)
print(numeros) # ['20', '30', '50']

texto = "Mi correo es roberto23@gmail.com"
patron = r"\w+@\w+\. \w+"
# Sustituimos correos electrónicos
texto_oculto = re.sub(patron, "[EMAIL OCULTO]", texto)
print(texto_oculto) # "Mi correo es [EMAIL OCULTO]"

texto = "Nombre: Juan, Edad: 30, País: España"
patron = r",\s*"
# Divide por comas seguidas de espacios
partes = re.split(patron, texto)
print(partes) # ['Nombre: Juan', 'Edad: 30', 'País: España']
```

En la documentación de Python se pueden consultar todos los métodos de cadena de texto que hay. Pincha en [este enlace](#) para verlos.

Table of Contents

Built-in Types

- Truth Value Testing
- Boolean Operations — **and**, **or**, **not**
- Comparisons
- Numeric Types — **int**, **float**, **complex**
 - Bitwise Operations on Integer Types
 - Additional Methods on Integer Types
 - Additional Methods on Float
 - Hashing of numeric types
- Boolean Type - **bool**
- Iterator Types
 - Generator Types
- Sequence Types — **list**, **tuple**, **range**
 - Common Sequence Operations
 - Immutable Sequence Types
 - Mutable Sequence Types
 - Lists
 - Tuples
 - Ranges
- Text Sequence Type — **str**
 - **String Methods**
 - **printf**-style String Formatting
- Binary Sequence Types — **bytes**, **bytearray**, **memoryview**

Datos categóricos

Los datos categóricos son valores que pertenecen a un conjunto limitado de categorías.
Por ejemplo:

- Género: "Masculino", "Femenino", "No Binario"
- Colores: "Rojo", "Azul", "Verde", "Amarillo", "Rosa"
- Clasificación de productos: "Básico", "Premium", "VIP"

Para su representación en pandas, se usa el tipo de dato Categorical que optimiza memoria y mejora el rendimiento.


```
import pandas as pd

df = pd.DataFrame({"Mes": ["Enero", "Marzo", "Mayo", "Marzo", "Abril"]})

# Convertimos la columna en tipo categórico
df["Mes"] = df["Mes"].astype("category")

print(df.dtypes)

           Mes    category
dtype: object
```

Al poner la columna como categórica se conseguimos un menor uso de memoria (útil en grandes datasets) y más rapidez en operaciones y filtros. Además, permite definir un orden en las categorías (Ejemplo: "Básico" < "Premium" < "VIP").

```
orden = ["Enero", "Febrero", "Marzo", "Abril", "Mayo", "Junio"]
df["Mes"] = pd.Categorical(df["Mes"], categories=orden, ordered=True)

#Ahora podemos hacer comparaciones
df["Mes"][2] > df["Mes"][4]
if True:
    print(df["Mes"][2], "es mayor que", df["Mes"][4])

           Mayo es mayor que Abril
```